Generalization and Regularization

Making deep networks perform well on unseen data

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

The generalization problem

Where we are

The last lectures gave us a network that **trains**:

$$f_\theta(x) \to \hat{y}, \quad \mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(x_i), y_i)$$

$$g_t = \nabla_\theta \mathcal{L}(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta\, g_t$$

Backprop gives the gradient; the optimizer drives $\mathcal{L}_{\text{train}}$ down. We can now make the **training loss** small.

But is low training loss the goal?

Q QUESTION

Does low training loss alone guarantee that a model will perform well on new data?

A. Yes; training loss is the only metric that matters

B. Yes, if the model has enough parameters

C. No; it may have memorized the training set

D. No; because gradient descent cannot reduce loss

Answer: training is not the goal

A ANSWER

Correct: C — It may have memorized the training set

Why: We care about expected performance on unseen data. A small training loss can coexist with a large generalization gap.

Train loss vs test loss

D DERIVATION

Training loss — an empirical average over the data we happened to collect:

$$\mathcal{L}_{\text{train}(\theta)} = \frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(x_i), y_i)$$

Test loss — the expectation over the **true** data distribution p_{data}:

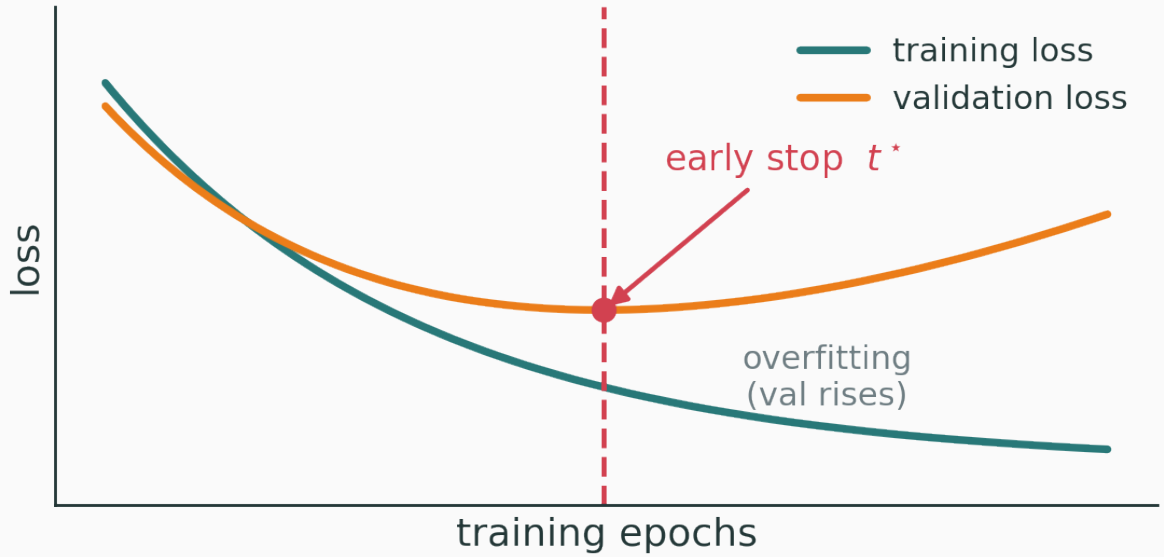
$$\mathcal{L}_{\text{test}(\theta)} = \mathbb{E}_{(x,y)\sim p_{\text{data}}}[\ell(f_\theta(x), y)]$$

We only ever **see finite data**; $\mathcal{L}_{\text{train}}$ is a noisy estimate of the quantity we actually want, $\mathcal{L}_{\text{test}}$.

The generalization gap

$$\underbrace{\mathcal{L}_{\text{test}} - \mathcal{L}_{\text{train}}}_{\text{generalization gap}}$$

Overfitting: training loss keeps falling while validation loss **turns up**.



Overfitting can memorize

Give a high-capacity net random labels $y \to y_\pi$ (a fixed permutation): it can still drive $\mathcal{L}_{\text{train}} \to 0$.

Validation remains at chance: fitting the training set is **not** evidence of a learned, general rule.

The plan for this lecture

We have networks that train. Now we must stop them from **memorizing**.

Lever	**What it controls**
splits + early stopping	when to stop
weight decay / ℓ_2	size of the weights
dropout	co-adaptation of units
data augmentation / mixup	invariance, linearity
label smoothing	overconfidence
implicit regularization	SGD noise, architecture, init

Data splits and early stopping

Three splits, three jobs

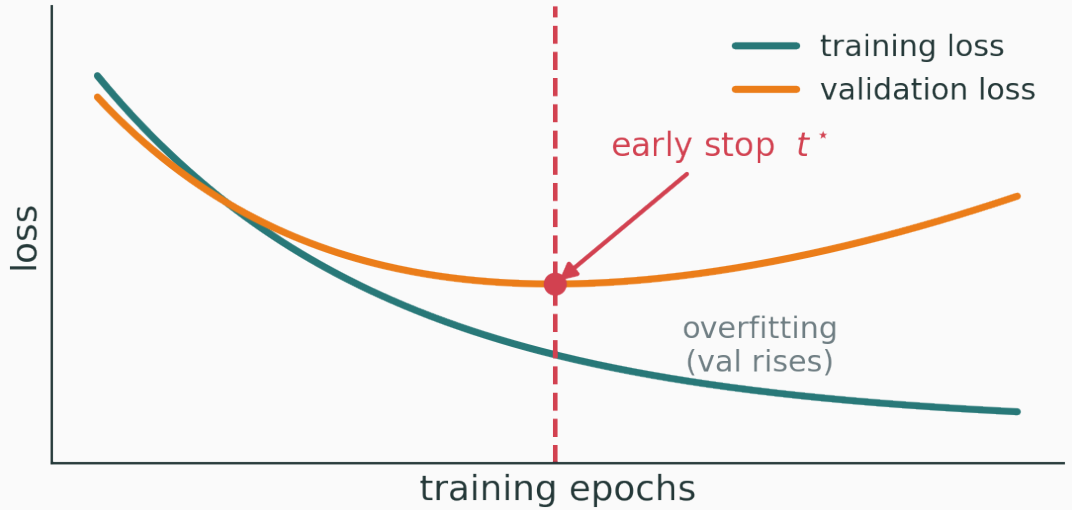
Split	Used to	Touches weights?
train	fit parameters θ	yes — via gradients
validation	select model / hyperparameters	no — only choices
test	final, unbiased estimate	no — used **once**

Never tune on the test set. Every peek leaks information and inflates your reported number.

Early stopping

Monitor validation loss each epoch; keep the **best** checkpoint, not the last:

$$t^\star = \arg\min_t \mathcal{L}_{\text{val}(\theta_t)}$$



train keeps falling; validation is U-shaped — stop at the bottom.

Worked example: early stopping

D DERIVATION

Six epochs of training. Which checkpoint do we keep?

epoch	1	2	3	4	5	6
$\mathcal{L}_{\text{train}}$	1.20	0.80	0.45	0.32	0.28	0.25
$\mathcal{L}_{\text{val}}$	1.25	0.98	0.82	**0.78**	0.84	0.97

- Training loss is **lowest at epoch 6** — but that is not what we want;
- Validation loss bottoms out at **epoch 4** (0.78), then rises → overfitting.

keep θ_4 — the last epoch is the wrong choice

Early stopping as implicit regularization

Why does stopping early help?

- Weights **grow** over training; stopping early keeps them **small**;
- fewer effective updates $\Longrightarrow$ a **simpler** function is explored;

Early stopping limits **effective capacity** without changing the architecture — a free, always-on regularizer. It is the cheapest one you have.

Interactive: train / val / overfit

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/double-descent

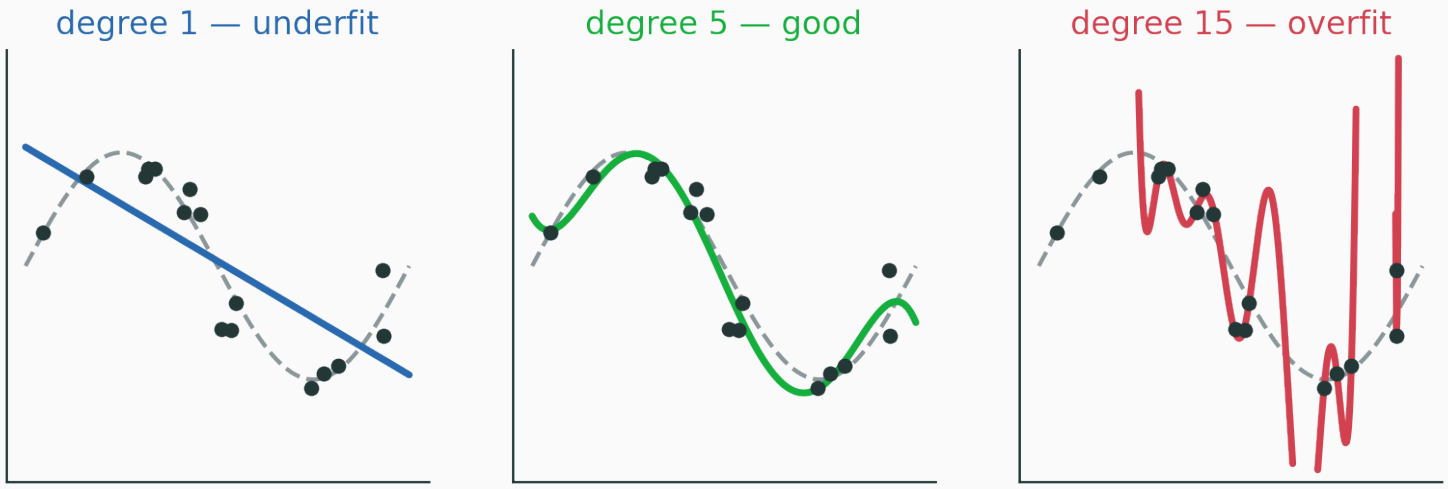
Sweep model capacity and training time and watch the train and validation curves separate — find the checkpoint where validation is lowest before the gap opens.

Find the checkpoint with the lowest **validation** loss rather than the last or lowest-training-loss checkpoint.

Capacity, bias and variance

Three regimes

Regime	$\mathcal{L}_{\text{train}}$	$\mathcal{L}_{\text{val}}$	Diagnosis
underfit	high	high	too little capacity
good fit	low	low	capacity ~ right
overfit	low	high	memorizing train



Bias–variance decomposition

Expected test error splits into three pieces:

$$\mathbb{E}[(\text{error})^2] \approx \underbrace{\text{bias}^2}_{\text{too simple}} + \underbrace{\text{variance}}_{\text{too sensitive}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

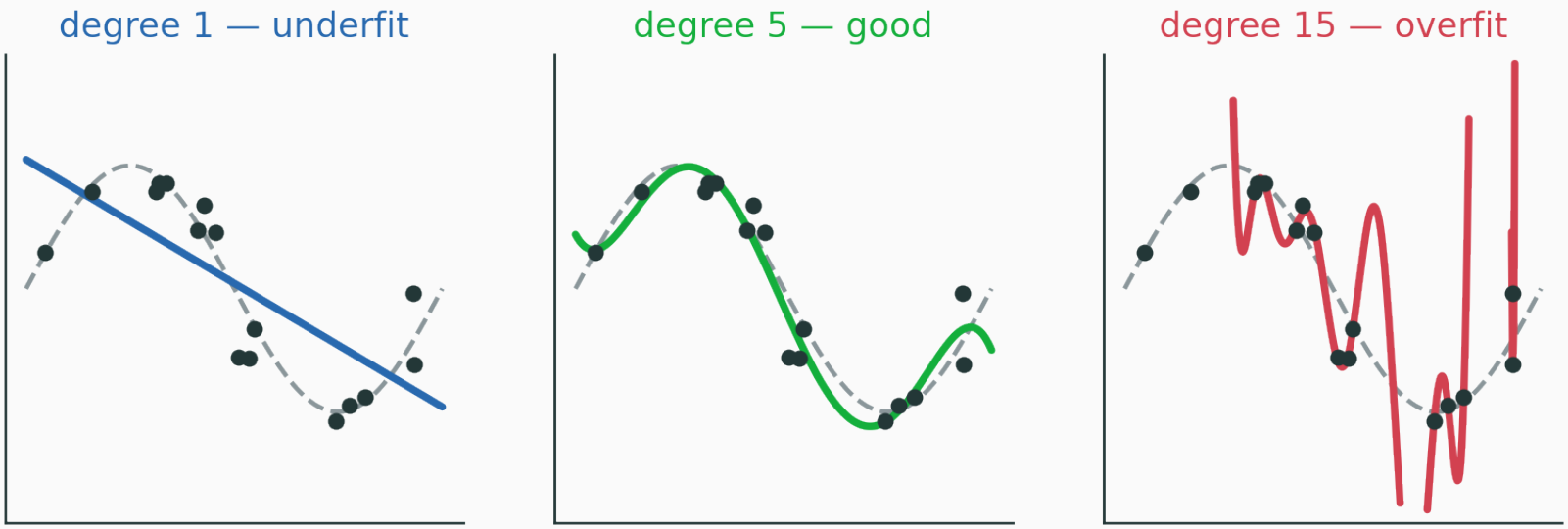
- **high bias** — the model is too simple; misses the true pattern (underfit);
- **high variance** — the model is too sensitive to the particular sample (overfit).

More capacity ↓ bias but ↑ variance. Classically we trade one against the other.

Worked example: polynomial fits

V VISUAL

Fit $y = \sin(2\pi x) + \varepsilon$ from noisy samples with polynomials of degree d:

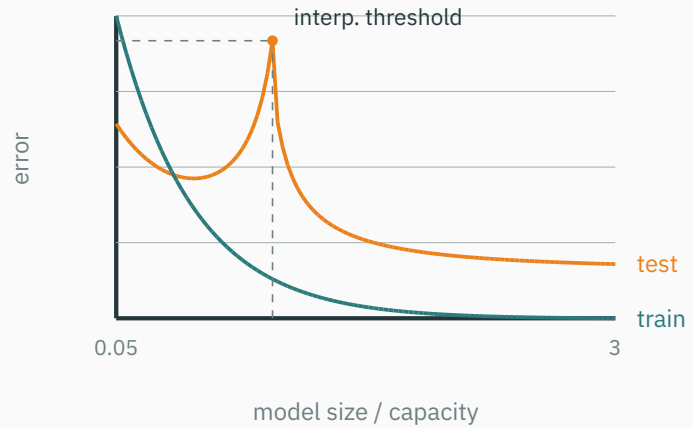


$d = 1$ underfits (high bias) · $d = 5$ fits · $d = 15$ overfits (high variance)

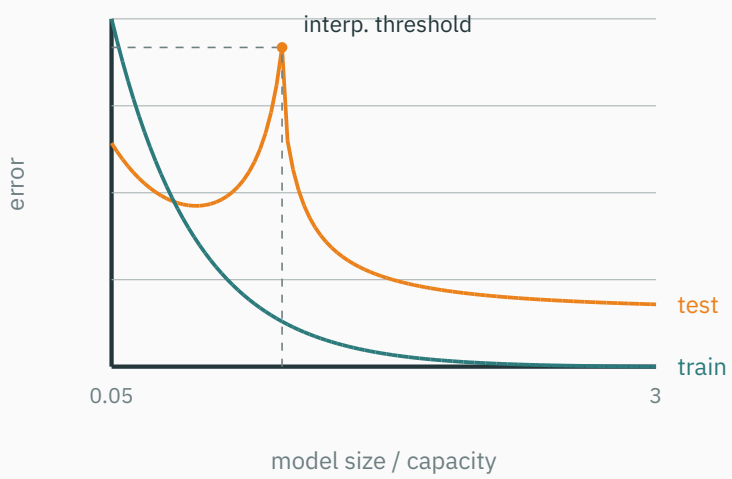
The deep-learning twist

Classical wisdom: **bigger model ⟹ more overfitting**. Modern practice complicates this.

- huge over-parameterized nets often generalize **well**, not worse;
- what governs generalization: **optimization, data, architecture, implicit bias** — not just parameter count;



Double descent



Past the interpolation threshold, test error can **fall again** — the modern "second descent".

> Parameter count is a poor proxy for capacity in deep nets. (active research)

But **validation discipline is still non-negotiable** — it is how we know what is happening.

Weight decay and ℓ_2 regularization

Penalize large weights

Add a penalty on the **size** of the weights to the data loss:

$$\mathcal{L}_{\text{reg}(\theta)} = \mathcal{L}_{\text{data}(\theta)} + \lambda \|\theta\|^2$$

- $\lambda > 0$ — the **regularization strength** (a hyperparameter);
- large weights $\Rightarrow$ sharp, wiggly functions; the penalty prefers **smooth** ones.

ℓ_2 regularization $\Leftrightarrow$ a **Gaussian prior** on the weights: MAP estimation with $\theta \sim \mathcal{N}(0, \sigma^2 I)$ gives exactly this penalty.

The gradient of the penalty

D DERIVATION

Differentiate $\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{data}} + \lambda \|\theta\|^2$:

$$\nabla \mathcal{L}_{\text{reg}} = \nabla \mathcal{L}_{\text{data}} + 2\lambda\, \theta$$

Substitute into the SGD step:

$$\theta_{t+1} = \theta_t - \eta(\nabla \mathcal{L}_{\text{data}} + 2\lambda\, \theta_t)$$

$$\theta_{t+1} = (1 - 2\eta\lambda)\, \theta_t - \eta\, \nabla \mathcal{L}_{\text{data}}$$

a multiplicative shrinkage $(1 - 2\eta\lambda)$ before the gradient step

Why “weight decay”?

Each step first **shrinks** the weight toward zero, then applies the gradient:

$$\theta_t \mapsto (1 - \eta\lambda')\,\theta_t$$

Absent any gradient, weights **decay** geometrically toward 0 — hence the name. ℓ_2 regularization and weight decay are the same mechanism (for SGD).

Worked example: one decay step

D DERIVATION

$\theta = 5$, gradient $g = 3$, learning rate $\eta = 0.1$, decay $\lambda' = 0.2$: **Without** weight decay — a plain gradient step:

$$\theta^{+} = \theta - \eta\, g = 5 - 0.1 \cdot 3 = 4.7$$

With weight decay, shrink first, then step: $\theta^{+} = (1 - \eta\lambda')\theta - \eta g$. Step 1 — the shrink factor: $1 - \eta\lambda' = 1 - 0.1 \cdot 0.2 = 0.98$. Step 2 — shrink the weight: $0.98 \cdot 5 = 4.9$. Step 3 — subtract the gradient step: $4.9 - 0.1 \cdot 3 = 4.9 - 0.3 = 4.6$.

the extra -0.1 (4.7 vs 4.6) is the shrinkage pulling the weight toward zero.

Adam ≠ Adam + ℓ_2: AdamW

D DERIVATION

For **SGD**, adding $\lambda\|\theta\|^2$ to the loss $\equiv$ weight decay (up to scaling). For **Adam** they **differ**: the $2\lambda\theta$ term gets divided by $\sqrt{\hat{v}}$ like any gradient — so decay is **distorted** per coordinate. Loshchilov & Hutter **decoupled** it — apply decay directly to the weights:

$$\theta \mapsto \mathrm{AdamStep}(\theta), \qquad \theta \mapsto (1 - \eta\lambda)\,\theta$$

use AdamW, not Adam with naive ℓ_2

When (and when not) to decay

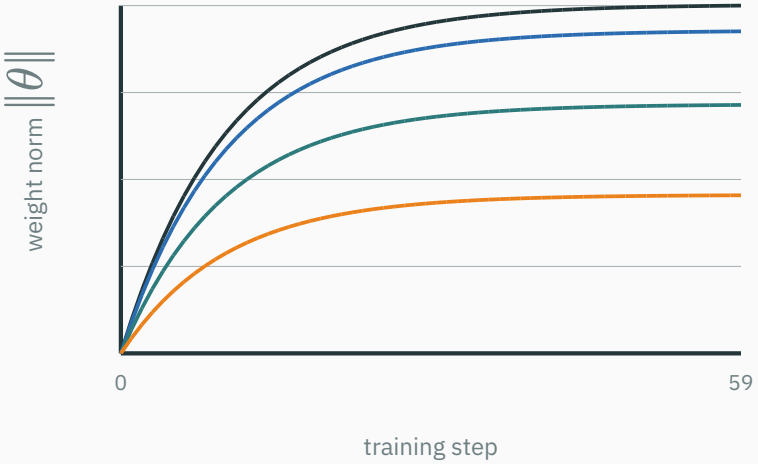
- Use it almost everywhere: **MLPs, CNNs, Transformers**, large models;
- typical λ: 10^{-4} to 10^{-2} (tune on validation);
- **do not** decay **biases**, **norm** scale/shift, and often not **embeddings** — decaying them helps nothing and can hurt.

Rule of thumb: decay the **weight matrices**, leave the **one-dimensional** parameters alone.

Weight decay shrinks the weights

V VISUAL

Weight norm $\|\theta\|$ over training for several λ:



$$\lambda = 0 \cdot \lambda = 0.01 \cdot \lambda = 0.05 \cdot \lambda = 0.15$$

larger $\lambda \Rightarrow$ smaller final weights $\Rightarrow$ smoother function.

Interactive: weight decay

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/weight-decay

sweep λ and watch the weight norm shrink and the decision boundary smooth out; too large and the model underfits.

Compare the unchanged shrink factor with the doubled data-gradient step when η changes.

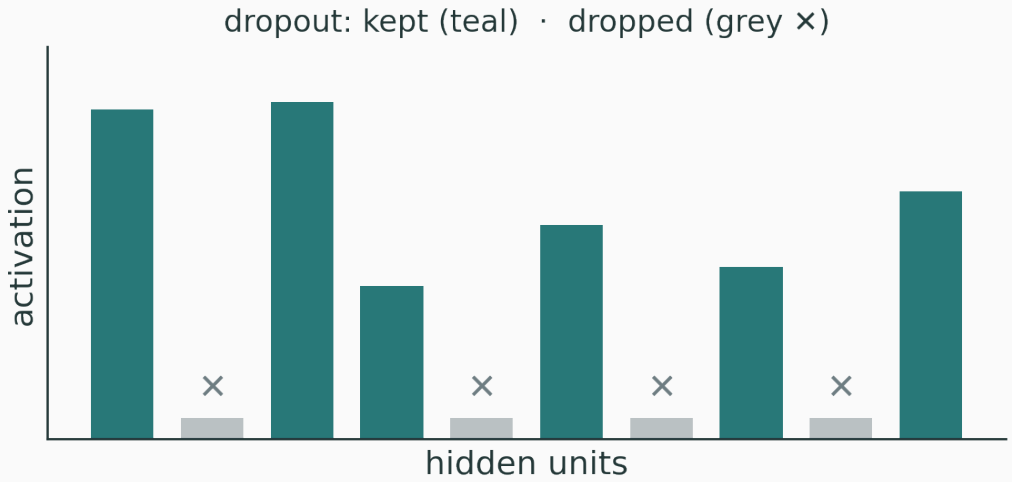
Dropout

The idea

During training, randomly **drop** each hidden unit with probability $1 - p$:

$$m_j \sim \text{Bernoulli}(p), \qquad \tilde{h} = m \odot h$$

- a different random **subnetwork** is trained on every minibatch;
- units cannot rely on any single partner $\Rightarrow$ less **co-adaptation**.



Inverted dropout: keep the scale

D DERIVATION

Dropping units shrinks the expected activation. **Rescale** by $\frac{1}{p}$ during training:

$$\tilde{h} = \frac{m \odot h}{p}$$

Then the expected activation is **unchanged**:

$$\mathbb{E}\left[\tilde{h}_j\right] = \frac{1}{p} \cdot \left(p \cdot h_j + (1-p) \cdot 0\right) = h_j$$

no test-time rescaling needed — at test time just use the full network

Worked example: inverted dropout

D DERIVATION

$h = [2, 4, 6, 8]$, keep prob $p = 0.5$, sampled mask $m = [1, 0, 1, 0]$: Step 1 — apply the mask (units 2 and 4 are dropped): $m \odot h = [2, 0, 6, 0]$. Step 2 — rescale survivors by $1/p = 2$:

$$\tilde{h} = \frac{m \odot h}{p} = \frac{[2, 0, 6, 0]}{0.5} = [4, 0, 12, 0]$$

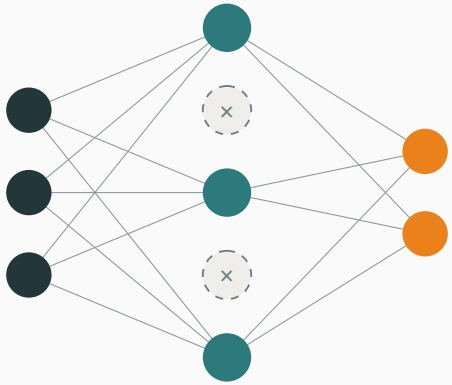
Check the expectation (each unit kept w.p. 0.5, scaled by 2):

$$\mathbb{E}\left[\tilde{h}\right] = 0.5 \cdot \left(\frac{h}{0.5}\right) = h = [2, 4, 6, 8]$$

surviving units are **doubled**, so the layer's expected output is preserved.

Why it helps: model averaging

Each mask trains a **different subnetwork** — dashed units are dropped this step:



We fit exponentially many subnetworks; at test the full net **approximates their average**.

An **ensemble-like** effect from one model — but an intuition, not an exact Bayesian average. Do not overclaim it.

Dropout caveats

- Works best on **fully-connected** layers and **classifier heads**;
- less useful alongside strong **augmentation** and **normalization** (which already regularize);
- adds gradient noise $\Rightarrow$ can **slow optimization**, needs more epochs;
- tune the **keep** rate $p \approx 0.5$–0.8 (i.e. drop 20–50%).

Interactive: dropout

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/dropout-playground

Toggle dropout on a small MLP, sweep the rate, and watch training vs validation loss — see how too much drop underfits and too little overfits.

Data augmentation

Regularize by enlarging the data

Generate new **label-preserving** examples from the ones you have:

$$(x, y) \mapsto (T(x), y), \qquad T \text{ preserves the label}$$

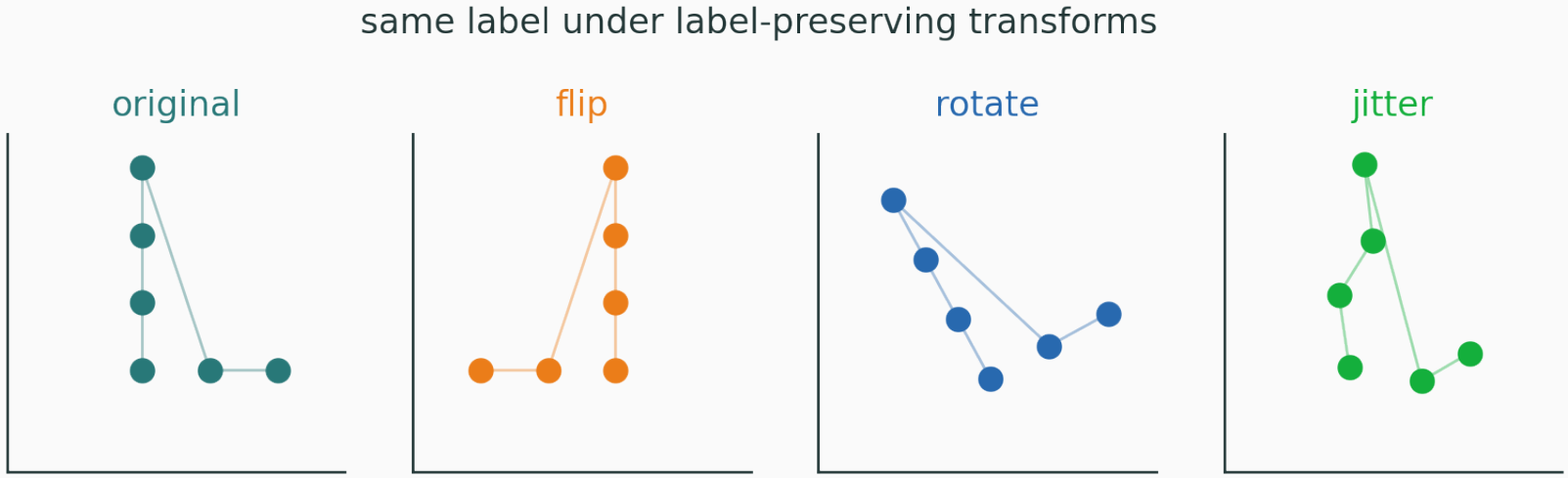
Domain	**Transforms T**
images	crop, flip, rotate, color jitter, noise
audio / time series	time-shift, mask spans, add noise
text	paraphrase, synonym swap, back-translation

Augmentation teaches invariance

V VISUAL

If T preserves the label, we want the model to be **invariant** to it:

$$f_\theta(x) \approx f_\theta(T(x))$$



same object, many views — the network learns to ignore the nuisance transform.

Bad augmentation encodes wrong knowledge

Augmentation **injects domain knowledge** — the wrong T injects the wrong knowledge:

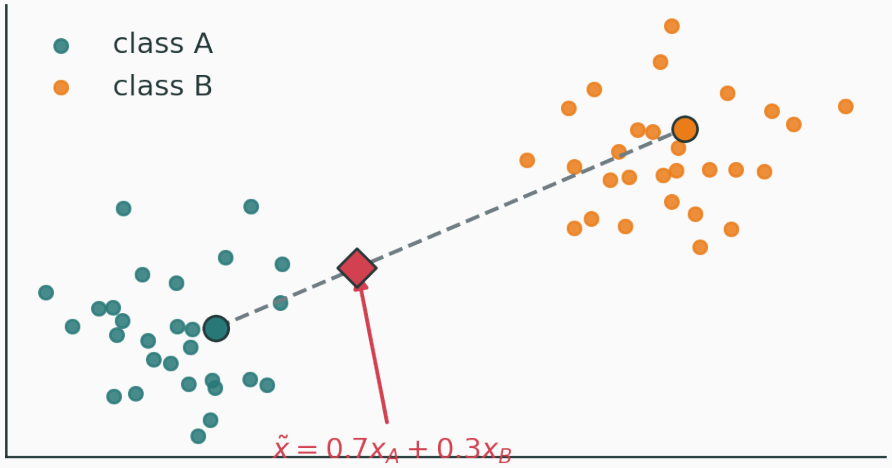
- flipping a **6** into a **9** — the label changed;
- **time-reversing** speech or a heartbeat — no longer valid;
- **color** jitter when **color is the label** (ripe vs unripe fruit);
- **cropping out** the object you are trying to classify.

T must preserve the label. Choosing T **is** a modelling decision about what should not matter.

Mixup: interpolate examples

Blend two examples **and** their labels with $\lambda \sim \mathrm{Beta}(\alpha, \alpha)$:

$$\tilde{x} = \lambda x_i + (1-\lambda) x_j, \qquad \tilde{y} = \lambda y_i + (1-\lambda) y_j$$



convex combinations encourage **linear** behaviour between examples.

Worked example: mixup

D DERIVATION

x_A = cat, $y_A = [1, 0]$; x_B = dog, $y_B = [0, 1]$; mix $\lambda = 0.7$: The complementary weight is $1 - \lambda = 0.3$. Blend the inputs:

$$\tilde{x} = 0.7\, x_A + 0.3\, x_B$$

Blend the labels the **same** way:

$$\tilde{y} = 0.7[1, 0] + 0.3[0, 1] = [0.7, 0.3]$$

Train with the **soft** cross-entropy against this target:

$$\ell = -\sum_k \tilde{y}_k \log p_k = -0.7 \log p_1 - 0.3 \log p_2$$

the network is asked to be 70% cat, 30% dog — no hard boundary jump.

Interactive: augmentation + mixup

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/augmentation-mixup

apply transforms and mixup to a 2-D dataset and watch the decision boundary become smoother and more invariant.

Observe how soft targets discourage infinite-margin, overconfident logits.

Label smoothing

Hard labels push to extremes

With a one-hot target, cross-entropy keeps pushing $p_y \to 1$:

$$\ell = -\log p_y \quad \Longrightarrow \quad \text{logits} \to \pm\infty$$

The model is rewarded for **ever more extreme** confidence — even on ambiguous or mislabelled examples. This encourages **overconfidence**.

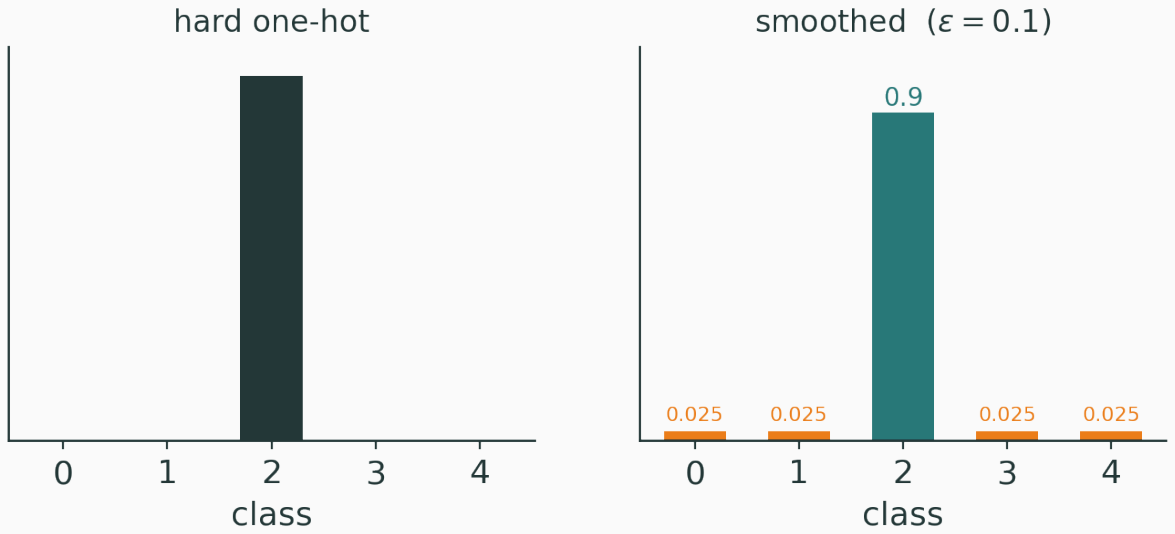
Softening the target

D DERIVATION

Move a little mass ε off the correct class onto the rest:

$$y_k^{\text{smooth}} = \begin{cases} 1-\varepsilon & \text{if } \; k = y \\ \varepsilon/(K-1) & \text{otherwise} \end{cases}$$

Equivalently, mix the one-hot with a uniform distribution over the **other** classes.



Worked example: label smoothing

D DERIVATION

$K = 5$ classes, smoothing $\varepsilon = 0.1$, correct class $= 2$: Correct class keeps most of the mass: $1 - \varepsilon = 0.9$. The remaining $\varepsilon = 0.1$ is split over the $K - 1 = 4$ others: $\frac{\varepsilon}{K-1} = \frac{0.1}{4} = 0.025$. Assemble the target vector:

$$y^{\text{smooth}} = [0.025, 0.025, 0.9, 0.025, 0.025]$$

Verify it is a distribution: $0.9 + 4 \cdot 0.025 = 1$.

still sums to 1; the target is achievable with **finite** logits.

Why it helps (and when it hurts)

- **discourages extreme confidence** — logits stay bounded;
- improves **calibration** (predicted probabilities match accuracy);
- adds **robustness to noisy labels** and often improves generalization;

Can **hurt** when you need exact confidences — e.g. **knowledge distillation** or when downstream code reads calibrated probabilities.

Interactive: label smoothing

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/calibration

Sweep ε and watch the reliability diagram — see over-confident predictions pull back toward the diagonal as smoothing increases.

Noise and implicit regularization

Noise is a regularizer

Many effective regularizers work by **injecting noise** during training:

- **SGD** minibatch sampling — noisy gradients;
- **dropout** — noisy activations;
- **augmentation** — noisy inputs;
- **stochastic depth**, **label / input noise**.

Noise prevents the network from building **brittle**, precise dependencies — it must be robust to the perturbation.

Explicit vs implicit regularization

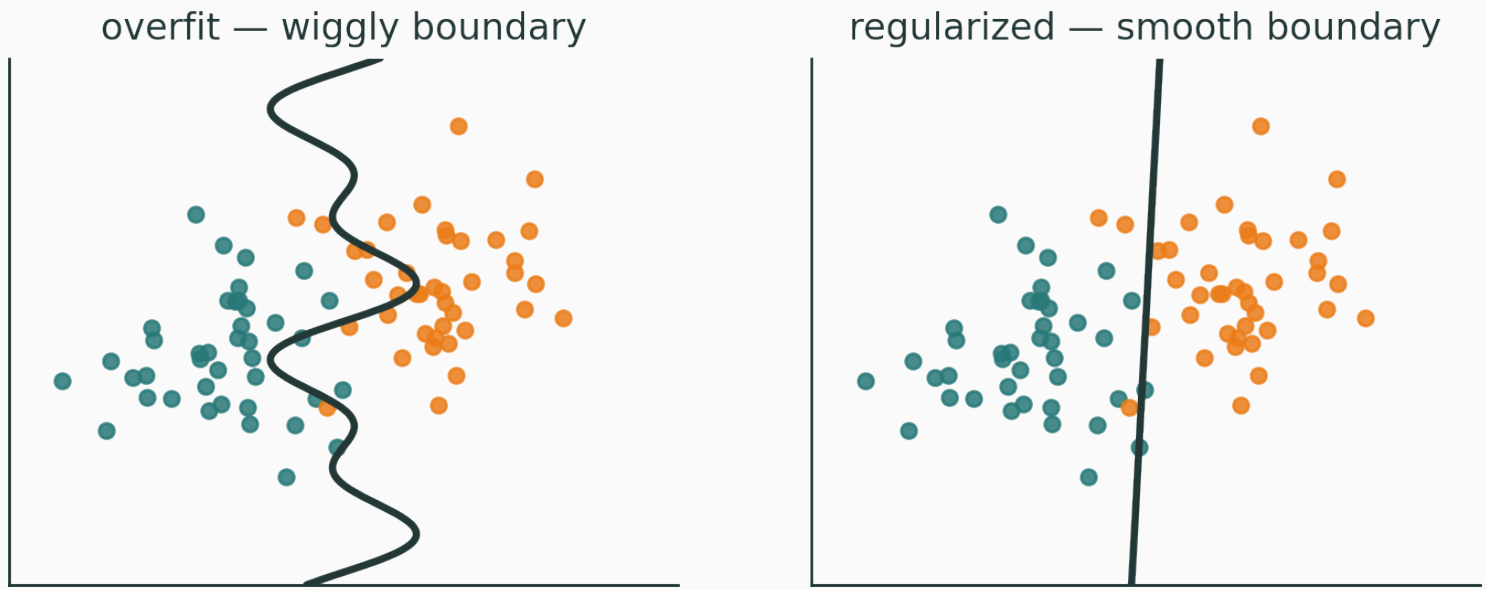
Explicit (a term / operation)	**Implicit** (a side effect)
$+\lambda\|\theta\|^2$ penalty	SGD gradient noise
dropout	architecture / optimizer bias
data augmentation	early stopping
label smoothing	normalization, initialization

Regularization is **not only a penalty term**.

Regularization changes the learned function

V VISUAL

Two models can both fit the training set to $\mathcal{L}_{\text{train}} \approx 0$ — yet differ:



regularization **picks** the smoother boundary — smaller norm, more invariance, less confidence, better val.

A practical workflow

First: debug before you regularize

Q QUESTION

Both training and validation loss are high. What should be tried before adding dropout?

A. Add stronger regularization immediately

B. Improve the fit: capacity, optimization, data checks, or training time

C. Stop training earlier

D. Increase label smoothing until training loss vanishes

Answer: diagnose underfitting first

A ANSWER

Correct: B — Improve the fit first

Why: High training loss signals underfitting or a pipeline issue. More regularization would make fitting even harder.

Fix the fit first: **bigger model**, **train longer**, **better LR**, **less augmentation**, **check the labels**.

Adding regularization to an underfit model makes it **worse**. Regularize only once you can **overfit**.

Then: regularize an overfit model

Training loss **low**, validation loss **high** ⟹ overfitting. Now reach for:

Lever	**First choice when…**
more data / augmentation	cheap to collect more or transform
weight decay	almost always — near-free
dropout	MLP / classifier head
early stopping	always on
smaller model	data is truly limited
label smoothing	over-confident / noisy labels

A regularization dashboard

Track more than the loss — watch **what** is being controlled:

- $\mathcal{L}_{\text{train}}$ **and** $\mathcal{L}_{\text{val}}$ (and the gap between them);
- $\text{acc}_{\text{train}}$ **and** acc_{val};
- weight norm $\|\theta\|$;
- confidence $\max_k p_k$ and **calibration error**.

If the **gap** widens while train keeps improving, you are overfitting — turn a regularization knob.

Worked diagnosis

D DERIVATION

Model	$\mathrm{acc}_{\mathrm{train}}$	$\mathrm{acc}_{\mathrm{val}}$	Action
A	65%	63%	underfit → **add capacity**, train longer
B	99%	72%	overfit → **regularize** / augment / early-stop

The gap is $\mathrm{acc}_{\mathrm{train}} - \mathrm{acc}_{\mathrm{val}}$: A $\to 2\%$, B $\to 27\%$.

- **A**: small gap, both low $\Longrightarrow$ the model **cannot fit** — do **not** regularize;
- **B**: large gap $\Longrightarrow$ the model **memorized** — rein it in.

Practical defaults

Setting	Default recipe
images (CNN)	augmentation + weight decay + early stopping
MLP / tabular	weight decay + dropout + early stopping
Transformer	AdamW + dropout + (label smoothing) + aug
small data	transfer learning + strong validation

Start simple, add one knob at a time, and let **validation** decide whether it helped.

Summary

One table: every method

Method	What it controls
early stopping	effective complexity — when to stop
weight decay	parameter norm $\|\theta\|$
dropout	co-adaptation / activation noise
data augmentation	invariance to nuisance transforms
mixup	linearity between examples
label smoothing	overconfidence / calibration
validation split	model selection — the judge

Final mental model — one idea

Optimization **reduces training loss**.
Regularization **improves unseen-data performance**.

validation tells us whether it is actually helping.

Optimization reduces train loss; regularization improves generalization; validation is the judge.

Next: **CNNs** — why fully-connected nets are inefficient for images.